

安装与配置

安装与配置

本章介绍 iSulad 的安装、安装后配置，以及升级和卸载的方法。

说明： iSulad 的安装、升级、卸载均需要使用 root 权限。

- 安装与配置
 - 安装方法
 - 配置方法

安装方法

iSulad 可以通过 yum 或 rpm 命令两种方式安装，由于 yum 会自动安装依赖，而 rpm 命令需要手动安装所有依赖，所以推荐使用 yum 安装。

这里给出两种安装方式的操作方法。

- （推荐）使用 yum 安装 iSulad，参考命令如下：

```
# sudo yum install -y iSulad
```

- 使用 rpm 安装 iSulad，需要下载 iSulad 及其所有依赖库的 RPM 包，然后手动安装。安装单个 iSulad 的 RPM 包（依赖包安装方式相同），参考命令如下：

```
# sudo rpm -ihv iSulad-xx.xx.xx-YYYYmmdd.HHMMSS.gitxxxxxxxx.aarch64.rpm
```

```
# isulad -l DEBUG
```

配置方法

iSulad 安装完成后，可以根据需要进行相关配置。

配置方式

轻量级容器引擎（iSulad）服务端 daemon 为 isulad，isulad 可以通过配置文件进行配置，也可以通过命令行的方式进行配置，例如：isulad --xxx，优先级从高到低是：命令行方式>配置文件>代码中默认配置。

说明： 如果采用 systemd 管理 iSulad 进程，修改/etc/sysconfig/iSulad 文件中的 OPTIONS 字段，等同于命令行方式进行配置。

- **命令行方式**

在启动服务的时候，直接通过命令行进行配置。其配置选项可通过以下命令查阅：

```
# isulad --help
lightweight container runtime daemon
```

Usage: isulad [global options]

GLOBAL OPTIONS:

--authorization-plugin	Use authorization plugin
--cgroup-parent	Set parent cgroup for all containers
--cni-bin-dir	The full path of the directory in which to search for CNI plugin binaries. Default: /opt/cni/bin
--cni-conf-dir	The full path of the directory in which to search for CNI config files. Default: /etc/cni/net.d
--default-ulimit	Default ulimits for containers (default [])
-e, --engine	Select backend engine
-g, --graph	Root directory of the iSulad runtime
-G, --group	Group for the unix socket(default is isulad)
--help	Show help
--hook-spec	Default hook spec file applied to all containers
-H, --host	The socket name used to create gRPC server
--image-layer-check	Check layer integrity when needed
--image-opt-timeout	Max timeout(default 5m) for image operation
--insecure-registry	Disable TLS verification for the given registry
--insecure-skip-verify-enforce	Force to skip the insecure verify(default false)
--log-driver	Set daemon log driver, such as: file
-l, --log-level	Set log level, the levels can be: FATAL ALERT CRIT ERROR WARN NOTICE INFO DEBUG TRACE
--log-opt	Set daemon log driver options, such as: log-path=/tmp/logs/ to set directory where to store daemon logs
--native.umask	Default file mode creation mask (umask) for containers
--network-plugin	Set network plugin, default is null, support null and cni
-p, --pidfile	Save pid into this file
--pod-sandbox-image	The image whose network/ipc namespaces

containers in each pod will use. (default "rnd-dockerhub.huawei.com/library/pause-
\${machine}:3.0")

--registry-mirrors Registry to be prepended when pulling unqualified
images, can be specified multiple times

--start-timeout timeout duration for waiting on a container to start
before it is killed

-S, --state Root directory for execution state files

--storage-driver Storage driver to use(default overlay2)

-s, --storage-opt Storage driver options

--tls Use TLS; implied by --tlsverify

--tlscacert Trust certs signed only by this CA (default
"/root/.iSulad/ca.pem")

--tlscert Path to TLS certificate file (default "/root/.iSulad/cert.pem")

--tlskey Path to TLS key file (default "/root/.iSulad/key.pem")

--tlsverify Use TLS and verify the remote

--use-decrypted-key Use decrypted private key by default(default true)

-V, --version Print the version

--websocket-server-listening-port CRI websocket streaming service listening port
(default 10350)

示例： 启动 isulad，并将日志级别调整成 DEBUG

```
# isulad -l DEBUG
```

- **配置文件方式**

isulad 配置文件为/etc/isulad/daemon.json，各配置字段说明如下：

配置参数

配置文件示例

参数解释

备注

-e, --engine

"engine": "lcr"

iSulad 的运行时，默认是 lcr

无

-G, --group

"group": "isulad"

socket 所属组

无

--hook-spec

"hook-spec": "/etc/default/isulad/hooks/default.json"

针对所有容器的默认钩子配置文件

无

-H, --host

"hosts": "unix:///var/run/isulad.sock"

通信方式

除本地 socket 外，还支持 tcp://ip:port 方式，port 范围（0-65535，排除被占用端口）

--log-driver

"log-driver": "file"

日志驱动配置

无

-l, --log-level

"log-level": "ERROR"

设置日志输出级别

无

--log-opt

"log-opts": { "log-file-mode": "0600", "log-path": "/var/lib/isulad", "max-file": "1", "max-size": "30KB" }

日志相关的配置

可以指定 max-file, max-size, log-path。max-file 指日志文件个数；max-size 指日志触发防爆的阈值，若 max-file 为 1, max-size 失效；log-path 指定日志文件存储路径；log-file-mode 用于设置日志文件的读写权限，格式要求必须为八进制格式，如 0666。

--start-timeout

"start-timeout": "2m"

启动容器的耗时

无

--runtime

"default-runtime": "lcr"

创建容器时的 runtime 运行时，默认是 lcr

当命令行和配置文件均未指定时，默认为 lcr，runtime 的三种指定方式优先级：命令行>配置文件>默认 lcr，当前支持 lcr、kata-runtime。

无

```
"runtimes": { "kata-runtime": { "path": "/usr/bin/kata-runtime", "runtime-args": [ "--kata-config",  
"/usr/share/defaults/kata-containers/configuration.toml" ] }}
```

启动容器时，通过此字段指定多 runtimes 配置，在此集合中的元素均为有效的启动容器的 runtime 运行时。

容器的 runtime 白名单，在此集合中的自定义 runtime 才是有效的。示例为以 kata-runtime 为例的配置。

-p, --pidfile

"pidfile": "/var/run/isulad.pid"

保存 pid 的文件

当启动一个容器引擎的时候不需要配置，当需要启动两个以上的容器引擎时才需要配置。

-g, --graph

"graph": "/var/lib/isulad"

iSulad 运行时的根目录

-S, --state

"state": "/var/run/isulad"

执行文件的根目录

--storage-driver

"storage-driver": "overlay2"

镜像存储驱动，默认为 overlay2

当前只支持 overlay2

-s, --storage-opt

"storage-opts": ["overlay2.override_kernel_check=true"]

镜像存储驱动配置选项

可使用的选项为： overlay2.override_kernel_check=true # 忽略内核版本检查

overlay2.size=\${size} # 设置 rootfs quota 限额为\${size}大小

overlay2.basesize=\${size} #等价于 overlay2.size

--image-opt-timeout

"image-opt-timeout": "5m"

镜像操作超时时间，默认为 5m

值为-1 表示不限制超时。

--registry-mirrors

"registry-mirrors": ["docker.io"]

镜像仓库地址

无

`--insecure-registry`

`"insecure-registries": []`

不使用 TLS 校验的镜像仓库

无

`--native.umask`

`"native.umask": "secure"`

容器 umask 策略，默认"secure"，normal 为不安全配置

设置容器 umask 值。支持配置空字符（使用默认值

0027）、"normal"、"secure"：normal # 启动的容器 umask 值为 0022 secure

启动的容器 umask 值为 0027（默认值）

`--pod-sandbox-image`

`"pod-sandbox-image": "rnd-dockerhub.huawei.com/library/pause-aarch64:3.0"`

pod 默认使用镜像，默认为"rnd-dockerhub.huawei.com/library/pause-\${machine}:3.0"

无

`--network-plugin`

`"network-plugin": ""`

指定网络插件，默认为空字符，表示无网络配置，创建的 sandbox 只有 loop 网卡。

支持 cni 和空字符，其他非法值会导致 isulad 启动失败。

`--cni-bin-dir`

`"cni-bin-dir": ""`

指定 cni 插件依赖的二进制的存储位置

默认为/opt/cni/bin

`--cni-conf-dir`

`"cni-conf-dir": ""`

指定 cni 网络配置文件的存储位置

默认为/etc/cni/net.d

`--image-layer-check=false`

`"image-layer-check": false`

开启镜像层完整性检查功能，设置为 true；关闭该功能，设置为 false。默认为关闭。

isulad 启动时会检查镜像层的完整性，如果镜像层被破坏，则相关的镜像不可用。isulad 进行镜像完整性校验时，无法校验内容为空的文件和目录，以及链接文件。因此若镜像因掉电导致上述类型文件丢失，isulad 的镜像数据完整性校验可能无法识别。isulad 版本变更时需要检查是否支持该参数，如果不支持，需要从配置文件中删除。

`--insecure-skip-verify-enforce=false`

`"insecure-skip-verify-enforce": false`

Bool 类型，是否强制跳过证书的主机名/域名验证，默认为 false。当设置为 true 时，为不安全配置，会跳过证书的主机名/域名验证

默认为 false（不跳过），注意：因 isulad 使用的 yajl json 解析库限制，若在/etc/isulad/daemon.json 配置文件中配置非 Bool 类型的其他符合 json 格式的值时，isulad 将使用默认值 false。

`--use-decrypted-key=true`

`"use-decrypted-key": true`

Bool 类型，指定是否使用不加密的私钥。指定为 true，表示使用不加密的私钥；指定为 false，表示使用的为加密后的私钥，即需要进行双向认证。

默认配置为 true(使用不加密的私钥)，注意：因 isulad 使用的 yajl json 解析库限制，若在/etc/isulad/daemon.json 配置文件中配置非 Bool 类型的其他符合 json 格式的值时，isulad 将使用默认值 true。

`--tls`

"tls":false

Bool 类型，是否使用 TLS

默认值为 false，仅用于-H tcp://IP:PORT 方式

--tlsverify

"tlsverify":false

Bool 类型，是否使用 TLS，并验证远程访问

仅用于-H tcp://IP:PORT 方式

--tlscacert --tlscert --tlskey

"tls-config": { "CAFile": "/root/.iSulad/ca.pem", "CertFile":
"/root/.iSulad/server-cert.pem", "KeyFile":"/root/.iSulad/server-key.pem" }

TLS 证书相关的配置

仅用于-H tcp://IP:PORT 方式

--authorization-plugin

"authorization-plugin": "authz-broker"

用户权限认证插件

当前只支持 authz-broker

--cgroup-parent

"cgroup-parent": "lxc/mycgroup"

字符串类型，容器默认 cgroup 父路径

指定 daemon 端容器默认的 cgroup 父路径，如果客户端指定了--cgroup-parent，以客户端参数为准。注意：如果启了一个容器 A，然后启一个容器 B，容器 B 的 cgroup 父路径指定为容器 A 的 cgroup 路径，在删除容器的时候需要先删除容器 B 再删除容器 A，否则会导致 cgroup 资源残留。

--default-ulimits

"default-ulimits": { "nofile": { "Name": "nofile", "Hard": 6400, "Soft": 3200 } }

ulimit 指定限制的类型，soft 值及 hard 值

指定限制的资源类型，如“nofile”。两个字段名字必须相同，即都为 nofile，否则会报错。Hard 指定的值需要大于等于 Soft。如果 Hard 字段或者 Soft 字段未设置，则默认该字段默认为 0。

--websocket-server-listening-port

"websocket-server-listening-port": 10350

设置 CRI websocket 流式服务侦听端口，默认端口号 10350

指定 CRI websocket 流式服务侦听端，如果客户端指定了 --websocket-server-listening-port，以客户端参数为准。端口范围 1024-49151

示例：

```
# cat /etc/isulad/daemon.json
{
  "group": "isulad",
  "default-runtime": "lcr",
  "graph": "/var/lib/isulad",
  "state": "/var/run/isulad",
  "engine": "lcr",
  "log-level": "ERROR",
  "pidfile": "/var/run/isulad.pid",
  "log-opts": {
    "log-file-mode": "0600",
    "log-path": "/var/lib/isulad",
    "max-file": "1",
    "max-size": "30KB"
  },
  "log-driver": "stdout",
  "hook-spec": "/etc/default/isulad/hooks/default.json",
  "start-timeout": "2m",
  "storage-driver": "overlay2",
  "storage-opts": [
    "overlay2.override_kernel_check=true"
  ],
  "registry-mirrors": [
    "docker.io"
  ],
}
```

```

    "insecure-registries": [
        "rnd-dockerhub.huawei.com"
    ],
    "pod-sandbox-image": "",
    "image-opt-timeout": "5m",
    "native.umask": "secure",
    "network-plugin": "",
    "cni-bin-dir": "",
    "cni-conf-dir": "",
    "image-layer-check": false,
    "use-decrypted-key": true,
    "insecure-skip-verify-enforce": false
}

```

须知： 默认配置文件/etc/isulad/daemon.json 仅供参考，请根据实际需要进行配置

存储说明

文件名

文件路径

内容

*

/etc/default/isulad/

存放 isulad 的 OCI 配置文件和钩子模板文件，文件夹下的配置文件权限设置为 0640，sysmonitor 检查脚本权限为 0550

*

/etc/isulad/

isulad 的默认配置文件和 seccomp 的默认配置文件

isulad.sock

/var/run/

管道通信文件，客户端和 isulad 的通信使用的 socket 文件

isulad.pid

/var/run/

存放 isulad 的 PID，同时也是一个文件锁防止启动多个 isulad 实例

*

/run/lxc/

文件锁文件，isula 运行过程创建的文件

*

/var/run/isulad/

实时通讯缓存文件，isulad 运行过程创建的文件

*

/var/run/isula/

实时通讯缓存文件，isula 运行过程创建的文件

*

/var/lib/lcr/

LCR 组件临时目录

*

/var/lib/isulad/

isulad 运行的根目录，存放创建的容器配置、日志的默认路径、数据库文件、mount 点等 /var/lib/isulad/mnt/：容器 rootfs 的 mount 点 /var/lib/isulad/engines/lcr/：存放 lcr 容器配置目录，每个容器一个目录（以容器名命名）

约束限制

- 高并发场景（并发启动 200 容器）下，glibc 的内存管理机制会导致内存空洞以及虚拟内存较大（例如 10GB）的问题。该问题是高并发场景下 glibc 内存管理机制的限制，而不是内存泄露，不会导致内存消耗无限增大。可以通过设置 MALLOC_ARENA_MAX 环境变量来减少虚拟内存的问题，而且可以增大减少物理内存的概率。但是这个环境变量会导致 iSulad 的并发性能下降，需要用户根据实际情况做配置。

参考实践情况，平衡性能和内存，可以设置 `MALLOC_ARENA_MAX` 为 4。（在 arm64 服务器上面对 iSulad 的性能影响在 10%以内）

配置方法：

1. 手动启动 iSulad 的场景，可以直接 `export MALLOC_ARENA_MAX=4`，然后再启动 iSulad 即可。
2. systemd 管理 iSulad 的场景，可以修改 `/etc/sysconfig/iSulad`，增加一条 `MALLOC_ARENA_MAX=4` 即可。

- 为 daemon 指定各种运行目录时的注意事项

以 `--root` 为例，当使用 `/new/path/` 作为 daemon 新的 Root Dir 时，如果 `/new/path/` 下已经存在文件，且目录或文件名与 iSulad 需要使用的目录或文件名冲突（例如：`engines`、`mnt` 等目录）时，iSulad 可能会更新原有目录或文件的属性，包括属主、权限等为自己的属主和权限。

所以，用户需要明白重新指定各种运行目录和文件，会对冲突目录、文件属性的影响。建议用户指定的新目录或文件为 iSulad 专用，避免冲突导致的文件属性变化以及带来的安全问题。

- 日志文件管理：

须知： 日志功能对接：iSulad 由 systemd 管理，日志也由 systemd 管理，然后传输给 rsyslogd。rsyslogd 默认会对写日志速度有限制，可以通过修改 `/etc/rsyslog.conf` 文件，增加 `"$imjournalRateLimitInterval 0"` 配置项，然后重启 rsyslogd 的服务即可。

- 命令行参数解析限制

使用 iSulad 命令行接口时，其参数解析方式与 docker 略有不同，对于命令行中带参数的 flag，不管使用长 flag 还是短 flag，只会将该 flag 后第一个空格或与 flag 直接相连接的 '=' 后的字符串作为 flag 的参数，具体如下：

1. 使用短 flag 时，与 “-” 连接的字符串中的每个字符都被当作短 flag（当有 = 号时，= 号后的字符串当成 = 号前的短 flag 的参数）。

`isula run -du=root busybox` 等价于 `isula run -du root busybox` 或 `isula run -d -u=root busybox` 或 `isula run -d -u root busybox`，当使用 `isula run -du:root` 时，由于 `-:` 不是有效的短 flag，因此会报错。前述的命令行也等价于 `isula run -ud root busybox`，但不推荐这种使用方式，可能带来语义困扰。

2. 使用长 flag 时，与 “--” 连接的字符串作为一个整体当成长 flag，若包含=号，则=号前的字符串为长 flag，=号后的为参数。

```
isula run --user=root busybox
```

等价于

```
isula run --user root busybox
```

- 启动一个 isulad 容器，不能够以非 root 用户进行 isula run -i/-t/-ti 以及 isula attach/exec 操作。
- iSulad 对接 OCI 容器时，仅支持 kata-runtime 启动 OCI 容器。

DAEMON 多端口的绑定

描述

daemon 端可以绑定多个 unix socket 或者 tcp 端口，并在这些端口上侦听，客户端可以通过这些端口和 daemon 端进行交互。

接口

用户可以在/etc/isulad/daemon.json 文件的 hosts 字段配置一个或者多个端口。当然用户也可以不指定 hosts。

```
{
  "hosts": [
    "unix:///var/run/isulad.sock",
    "tcp://localhost:5678",
    "tcp://127.0.0.1:6789"
  ]
}
```

用户也可以在/etc/sysconfig/iSulad 中通过-H 或者--host 配置端口。用户同样可以不指定 hosts。

```
OPTIONS='-H unix:///var/run/isulad.sock --host tcp://127.0.0.1:6789'
```

如果用户在 daemon.json 文件及 iSulad 中均未指定 hosts，则 daemon 在启动之后将默认侦听 unix:///var/run/isulad.sock。

限制

- 用户不可以在/etc/isulad/daemon.json 和/etc/sysconfig/iSuald 两个文件中同时指定 hosts，如果这样做将会出现错误，isulad 无法正常启动；

unable to configure the isulad with file /etc/isulad/daemon.json: the following directives are specified both as a flag and in the configuration file: hosts: (from flag: [unix:///var/run/isulad.sock tcp://127.0.0.1:6789], from file: [unix:///var/run/isulad.sock tcp://localhost:5678 tcp://127.0.0.1:6789])

- 若指定的 host 是 unix socket，则必须是合法的 unix socket，需要以"unix://"开头，后跟合法的 socket 绝对路径；
- 若指定的 host 是 tcp 端口，则必须是合法的 tcp 端口，需要以"tcp://"开头，后跟合法的 IP 地址和端口，IP 地址可以为 localhost；
- 可以指定至多 10 个有效的端口，超过 10 个则会出现错误，isulad 无法正常启动。

配置 TLS 认证与开启远程访问

描述

iSulad 采用 C/S 模式进行设计，在默认情况，iSulad 守护进程 isulad 只侦听本地/var/run/isulad.sock，因此只能在本机通过客户端 isula 执行相关命令操作容器。为了能使 isula 可以远程访问容器，isulad 守护进程需要通过 tcp:ip 的方式侦听远程访问的端口。然而，仅通过简单配置 tcp ip:port 进行侦听，这样会导致所有的 ip 都可以通过调用 isula -H tcp://:port 与 isulad 通信，容易导致安全问题，因此推荐使用较安全版本的 TLS(**Transport Layer Security - 安全传输层协议**) 方式进行远程访问。

生成 TLS 证书

- 明文私钥和证书生成方法示例

```
#!/bin/bash
set -e
echo -n "Enter pass phrase:"
read password
echo -n "Enter public network ip:"
read publicip
echo -n "Enter host:"
read HOST
```

```
echo " => Using hostname: $publicip, You MUST connect to iSulad using this host!"
```

```
mkdir -p $HOME/.iSulad  
cd $HOME/.iSulad  
rm -rf $HOME/.iSulad/*
```

```
echo " => Generating CA key"  
openssl genrsa -passout pass:$password -aes256 -out ca-key.pem 4096  
echo " => Generating CA certificate"  
openssl req -passin pass:$password -new -x509 -days 365 -key ca-key.pem -sha256 -  
out ca.pem -subj  
"/C=CN/ST=zhejiang/L=hangzhou/O=Huawei/OU=iSulad/CN=iSulad@huawei.com"  
echo " => Generating server key"  
openssl genrsa -passout pass:$password -out server-key.pem 4096  
echo " => Generating server CSR"  
openssl req -passin pass:$password -subj /CN=$HOST -sha256 -new -key server-  
key.pem -out server.csr  
echo subjectAltName = DNS:$HOST,IP:$publicip,IP:127.0.0.1 >> extfile.cnf  
echo extendedKeyUsage = serverAuth >> extfile.cnf  
echo " => Signing server CSR with CA"  
openssl x509 -req -passin pass:$password -days 365 -sha256 -in server.csr -CA  
ca.pem -CAkey ca-key.pem -CAcreateserial -out server-cert.pem -extfile extfile.cnf  
echo " => Generating client key"  
openssl genrsa -passout pass:$password -out key.pem 4096  
echo " => Generating client CSR"  
openssl req -passin pass:$password -subj '/CN=client' -new -key key.pem -out  
client.csr  
echo " => Creating extended key usage"  
echo extendedKeyUsage = clientAuth > extfile-client.cnf  
echo " => Signing client CSR with CA"  
openssl x509 -req -passin pass:$password -days 365 -sha256 -in client.csr -CA  
ca.pem -CAkey ca-key.pem -CAcreateserial -out cert.pem -extfile extfile-client.cnf  
rm -v client.csr server.csr extfile.cnf extfile-client.cnf  
chmod -v 0400 ca-key.pem key.pem server-key.pem  
chmod -v 0444 ca.pem server-cert.pem cert.pem
```

- 加密私钥和证书请求文件生成方法示例

```
#!/bin/bash
```



```
echo -n "Enter public network ip:"
read publicip
echo -n "Enter pass phrase:"
read password

# remove certificates from previous execution.
rm -f *.pem *.srl *.csr *.cnf

# generate CA private and public keys
echo 01 > ca.srl
openssl genrsa -aes256 -out ca-key.pem -passout pass:$password 2048
openssl req -subj
'/C=CN/ST=zhejiang/L=hangzhou/O=Huawei/OU=iSulad/CN=iSulad@huawei.com' -
new -x509 -days $DAYS -passin pass:$password -key ca-key.pem -out ca.pem

# create a server key and certificate signing request (CSR)
openssl genrsa -aes256 -out server-key.pem -passout pass:$PASS 2048
openssl req -new -key server-key.pem -out server.csr -passin pass:$password -subj
'/CN=iSulad'

echo subjectAltName = DNS:iSulad,IP:${publicip},IP:127.0.0.1 > extfile.cnf
echo extendedKeyUsage = serverAuth >> extfile.cnf
# sign the server key with our CA
openssl x509 -req -days $DAYS -passin pass:$password -in server.csr -CA ca.pem -
CAkey ca-key.pem -out server-cert.pem -extfile extfile.cnf

# create a client key and certificate signing request (CSR)
openssl genrsa -aes256 -out key.pem -passout pass:$password 2048
openssl req -subj '/CN=client' -new -key key.pem -out client.csr -passin pass:
$password

# create an extensions config file and sign
echo extendedKeyUsage = clientAuth > extfile.cnf
openssl x509 -req -days 365 -passin pass:$password -in client.csr -CA ca.pem -CAkey
ca-key.pem -out cert.pem -extfile extfile.cnf

# remove the passphrase from the client and server key
openssl rsa -in server-key.pem -out server-key.pem -passin pass:$password
openssl rsa -in key.pem -out key.pem -passin pass:$password
```

```
# remove generated files that are no longer required
rm -f ca-key.pem ca.srl client.csr extfile.cnf server.csr
```

接口

```
{
  "tls": true,
  "tls-verify": true,
  "tls-config": {
    "CAFile": "/root/.iSulad/ca.pem",
    "CertFile": "/root/.iSulad/server-cert.pem",
    "KeyFile": "/root/.iSulad/server-key.pem"
  }
}
```

限制

服务端支持的模式如下：

- 模式 1（验证客户端）：tlsverify, tlscacert, tlscert, tlskey。
- 模式 2（不验证客户端）：tls, tlscert, tlskey。

客户端支持的模式如下：

- 模式 1(使用客户端证书进行身份验证，并根据给定的 CA 验证服务器)：tlsverify, tlscacert, tlscert, tlskey。
- 模式 2(验证服务器)：tlsverify, tlscacert。

如果需要采用双向认证方式进行通讯，则服务端采用模式 1，客户端采用模式 1；

如果需要采用单向认证方式进行通讯，则服务端采用模式 2，客户端采用模式 2。

须知：

- 采用 RPM 安装方式时，服务端配置可通过/etc/isulad/daemon.json 以及/etc/sysconfig/iSulad 配置修改
- 相比非认证或者单向认证方式，双向认证具备更高的安全性，推荐使用双向认证的方式进行通讯
- GRPC 开源组件日志不由 iSulad 进行接管，如果需要查看 GRPC 相关日志，请按需设置 GRPC_VERBOSEITY 和 GRPC_TRACE 环境变量

示例

服务端：

```
isulad -H=tcp://0.0.0.0:2376 --tlsverify --tlscacert ~/.iSulad/ca.pem --tlscert  
~/.iSulad/server-cert.pem --tlskey ~/.iSulad/server-key.pem
```

客户端：

```
isula version -H=tcp://$HOSTIP:2376 --tlsverify --tlscacert ~/.iSulad/ca.pem --tlscert  
~/.iSulad/cert.pem --tlskey ~/.iSulad/key.pem
```

配置 **devicemapper** 存储驱动

使用 **devicemapper** 存储驱动需要先配置一个 **thinpool** 设备，而配置 **thinpool** 需要一个独立的块设备，且该设备需要有足够的空闲空间用于创建 **thinpool**，请用户根据实际需求确定。这里假设独立块设备为 **/dev/xvdf**，具体的配置方法如下：

一、配置 **thinpool**

1. 停止 **isulad** 服务。

```
# systemctl stop isulad
```

2. 基于块设备创建一个 **lvm** 卷。

```
# pvcreate /dev/xvdf
```

3. 使用刚才创建的物理卷创建一个卷组。

```
# vgcreate isula /dev/xvdf  
Volume group "isula" successfully created:
```

4. 创建名为 **thinpool** 和 **thinpoolmeta** 的两个逻辑卷。

```
# lvcreate --wipesignatures y -n thinpool isula -l 95%VG  
Logical volume "thinpool" created.
```

```
# lvcreate --wipesignatures y -n thinpoolmeta isula -l 1%VG  
Logical volume "thinpoolmeta" created.
```

5. 将新创建的两个逻辑卷转换成 **thinpool** 以及 **thinpool** 所使用的 **metadata**，这样就完成了 **thinpool** 配置。

```
# lvconvert -y --zero n -c 512K --thinpool isula/thinpool --poolmetadata  
isula/thinpoolmeta
```

WARNING: Converting logical volume isula/thinpool and isula/thinpoolmeta to
thin pool's data and metadata volumes with metadata wiping.
THIS WILL DESTROY CONTENT OF LOGICAL VOLUME (filesystem etc.)
Converted isula/thinpool to thin pool.

二、修改 isulad 配置文件

1. 如果环境之前运行过 isulad，请先备份之前的数据。

```
# mkdir /var/lib/isulad.bk  
# mv /var/lib/isulad/* /var/lib/isulad.bk
```

2. 修改配置文件

这里提供了两种配置方式，用户可根据实际情况的选择合适的方式。

- 编辑/etc/isulad/daemon.json，配置 storage-driver 字段值为 devicemapper，并配置 storage-opts 字段的相关参数，支持参数请参见参数说明。配置参考如下所示：

```
{  
  "storage-driver": "devicemapper"  
  "storage-opts": [  
    "dm.thinpooldev=/dev/mapper/isula-thinpool",  
    "dm.fs=ext4",  
    "dm.min_free_space=10%"  
  ]  
}
```

- 或者也可以通过编辑/etc/sysconfig/iSulad，在 isulad 启动参数里显式指定，支持参数请参见参数说明。配置参考如下所示：

```
OPTIONS="--storage-driver=devicemapper --storage-opt  
dm.thinpooldev=/dev/mapper/isula-thinpool --storage-opt dm.fs=ext4 --  
storage-opt dm.min_free_space=10%"
```

3. 启动 isulad，使配置生效。

```
# systemctl start isulad
```

参数说明

storage-opts 支持的参数请参见表 1。

表 1 storage-opts 字段参数说明

参数

是否必选

含义

dm.fs

是

用于指定容器使用的文件系统类型。当前必须配置为 ext4，即 dm.fs=ext4

dm.basesize

否

用于指定单个容器的最大存储空间大小，单位为 k/m/g/t/p，也可以使用大写字母，例如 dm.basesize=50G。该参数只在首次初始化时有效。

dm.mkfsarg

否

用于在创建基础设备时指定额外的 mkfs 参数。例如 “dm.mkfsarg=-O ^has_journal”

dm.mountopt

否

用于在挂载容器时指定额外的 mount 参数。例如 dm.mountopt=nodiscard

dm.thinpooldev

否

用于指定容器/镜像存储时使用的 thinpool 设备。

dm.min_free_space

否

用于指定最小的预留空间，用百分比表示。例如 `dm.min_free_space=10%`，表示当剩余存储空间只剩 10% 左右时，创建容器等和存储相关操作就会失败。

注意事项

- 配置 `devicemapper` 时，如果系统上没有足够的空间给 `thinpool` 做自动扩容，请禁止自动扩容功能。

禁止自动扩容的方法是把 `/etc/lvm/profile/isula-thinpool.profile` 中 `thin_pool_autoextend_threshold` 和 `thin_pool_autoextend_percent` 两个值都改成 100，如下所示：

```
activation {
  thin_pool_autoextend_threshold=100
  thin_pool_autoextend_percent=100
}
```

- 使用 `devicemapper` 时，容器文件系统必须配置为 `ext4`，需要在 `isulad` 的配置参数中加上 `--storage-opt dm.fs=ext4`。
- 当 `graphdriver` 为 `devicemapper` 时，如果 `metadata` 文件损坏且不可恢复，需要人工介入恢复。禁止直接操作或篡改 `daemon` 存储 `devicemapper` 的元数据。
- 使用 `devicemapper lvm` 时，异常掉电导致的 `devicemapper thinpool` 损坏，无法保证 `thinpool` 损坏后可以修复，也不能保证数据的完整性，需重建 `thinpool`。

iSula 开启了 `user namespace` 特性，切换 `devicemapper` 存储池时的注意事项

- 一般启动容器时，`deviceset-metadata` 文件为：`/var/lib/isulad/devicemapper/metadata/deviceset-metadata`。
- 使用了 `user namespace` 场景下，`deviceset-metadata` 文件使用的是：`/var/lib/isulad/{userNSUID.GID}/devicemapper/metadata/deviceset-metadata`。
- 使用 `devicemapper` 存储驱动，容器在 `user namespace` 场景和普通场景之间切换时，需要将对应 `deviceset-metadata` 文件中的 `BaseDeviceUUID` 内容清空；针对 `thinpool` 扩容或者重建的场景下，也同样的需要将对应 `deviceset-metadata` 文件中的 `BaseDeviceUUID` 内容清空，否则 `isulad` 服务会重启失败。